

Topic 1 [STL: stack class]

Problem statement: Write a program to create the following menu and manipulate Stack using **stack** class. Use the following methods to write the program. Also display appropriate message when required.

- i) **push()** -method to push data
- ii) **pop()** -method to pop data
- iii) **top()** -method to display top element
- iv) **empty()** -method to check whether stack is empty or not

```
**** Stack Menu ****
1. Push
2. Pop
3. Display
4. Exit
Enter your option:
```

Code	Output
<pre>#include<bits/stdc++.h> using namespace std; #define ll long long int #define str string #define newl cout<<endl #define clr system("cls"); class Stack { private: stack<int>ax; public: Stack() { } void Push(int x) { ax.push(x); } void Pop() { ax.pop(); } void Display() { if (ax.size()==0) { cout<<"NO ELEMENT";newl; return; } cout<<ax.top();newl; } }; void menu() { cout<<"**** Stack Menu ****\n"; cout<<" "<<"1. Push\n"; cout<<" "<<"2. Pop\n"; cout<<" "<<"3. Display\n"; cout<<" "<<"4. Exit\n"; cout<<" "<<"Enter your option: "; } int main() { Stack s; menu(); while(1)</pre>	<pre>#Main Menu: **** Stack Menu **** 1. Push 2. Pop 3. Display 4. Exit Enter your option: #Push operation: **** Stack Menu **** 1. Push 2. Pop 3. Display 4. Exit Enter your option: 1 Enter Value: 12 #Pop operation: **** Stack Menu **** 1. Push 2. Pop 3. Display 4. Exit Enter your option: 2 Value removed successfully Press Enter #Display operation: **** Stack Menu **** 1. Push 2. Pop 3. Display 4. Exit Enter your option: 3 20</pre>

<pre> { int a; cin>>a; if(a==1) { cout<<"Enter Value: "; int x;cin>>x; s.Push(x); clr menu(); } else if(a==2) { s.Pop(); cout<< "Value removed successfully\nPress Enter"; getchar();getchar(); clr menu(); } else if(a==3) { s.Display(); getchar();getchar(); clr menu(); } else { break; } } } </pre>	
--	--

Topic 2 [STL: queue class]

Problem statement: Write a program to create the following menu to manipulate Queue using queue class. Use the following methods to write the program. Also display appropriate message when required.

- i) **push()** -method to push data
- ii) **pop()** -method to pop data
- iii) **front()** -method to display front element
- iv) **back()** -method to display rear element
- v) **empty()** -method to check whether queue is empty or not

```

**** Queue Menu ****
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your option:

```

Code	Output
<pre> #include<bits/stdc++.h> using namespace std; #define _easy_ ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0); #define ll long long int #define str string #define newl cout<<endl #define clr system("cls"); class Queue { private: queue<int>ax; </pre>	<pre> #Main Menu: **** Queue Menu **** 1. Enqueue 2. Dequeue 3. Display 4. Exit Enter your option: </pre>

<pre> public: Queue() { } void Enqueue(int x) { ax.push(x); } void Dequeue() { ax.pop(); } void Display() { if (ax.size()==0) { cout<<"NO ELEMENT";newl; return; } cout<<"Front Element: "<<ax.front();newl; cout<<"Rear Element: "<<ax.back();newl; } }; void menu() { cout<<"**** Queue Menu ****\n"; cout<<" "<<"1. Enqueue\n"; cout<<" "<<"2. Dequeue\n"; cout<<" "<<"3. Display\n"; cout<<" "<<"4. Exit\n"; cout<<" "<<"Enter your option: "; } int main() { Queue q; menu(); while(1) { int a; cin>>a; if(a==1) { cout<<"Enter Value: "; int x;cin>>x; q.Enqueue(x); clr menu(); } else if(a==2) { q.Dequeue(); cout<<"Value removed successfully\nPress Enter"; getchar();getchar(); clr menu(); } else if(a==3) { q.Display(); getchar();getchar(); clr menu(); } else { break; } } } </pre>	#Enqueue operation: **** Queue Menu **** 1. Enqueue 2. Dequeue 3. Display 4. Exit Enter your option: 1 Enter Value: 15 #Dequeue operation: **** Queue Menu **** 1. Enqueue 2. Dequeue 3. Display 4. Exit Enter your option: 2 Value removed successfully Press Enter #Display operation: **** Queue Menu **** 1. Enqueue 2. Dequeue 3. Display 4. Exit Enter your option: 3 Front Element: 25 Rear Element: 45
---	--

<pre> } } </pre>	
------------------	--

Topic 3 [STL: deque class]

Problem statement: Extend the above menu with more features using **deque** class. Use the methods to given in the lecture note for **deque** class.

Code	Output
<pre> #include<bits/stdc++.h> using namespace std; #define _easy_ ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0); #define ll long long int #define str string #define P_B push_back #define p_b pop_back #define P_F push_front #define p_f pop_front #define newl cout<<endl #define clr system("cls"); class Deque { private: deque<int>ax; public: Deque() { } void Push_Back(int x) { ax.P_B(x); } void Push_Front(int x) { ax.P_F(x); } void Pop_Back() { ax.p_b(); } void Pop_Front() { ax.p_f(); } void Display() { if (ax.size()==0) { cout<<"NO ELEMENT";newl; return; } cout<<"Front Element: "<<ax.front();newl; cout<<"Rear Element: "<<ax.back();newl; } }; void menu() { cout<<"**** Deque Menu ****\n"; cout<<" "<<"1. Enqueue (Front)\n"; cout<<" "<<"2. Enqueue (Back)\n"; cout<<" "<<"3. Dequeue (Front)\n"; cout<<" "<<"4. Dequeue (Back)\n"; cout<<" "<<"5. Display\n"; </pre>	<pre> #Main Menu: **** Deque Menu **** 1. Enqueue (Front) 2. Enqueue (Back) 3. Dequeue (Front) 4. Dequeue (Back) 5. Display 6. Exit Enter your option: #Enqueue Front operation: **** Deque Menu **** 1. Enqueue (Front) 2. Enqueue (Back) 3. Dequeue (Front) 4. Dequeue (Back) 5. Display 6. Exit Enter your option: 1 Enter Value: 15 #Enqueue Back operation: **** Deque Menu **** 1. Enqueue (Front) 2. Enqueue (Back) 3. Dequeue (Front) 4. Dequeue (Back) 5. Display 6. Exit Enter your option: 2 Enter Value: 20 #Dequeue Front operation: **** Deque Menu **** 1. Enqueue (Front) 2. Enqueue (Back) 3. Dequeue (Front) 4. Dequeue (Back) 5. Display 6. Exit Enter your option: 3 Value removed successfully from front Press Enter </pre>

```

cout<<"  "<<"6. Exit\n";
cout<<"  "<<"Enter your option: ";
}
int main()
{
    Deque dq;
    menu();
    while(1)
    {
        int a;
        cin>>a;
        if(a==1)
        {
            cout<<"Enter Value: ";
            int x;cin>>x;
            dq.Push_Front(x);
            clr
            menu();
        }
        else if(a==2)
        {
            cout<<"Enter Value: ";
            int x;cin>>x;
            dq.Push_Back(x);clr
            menu();
        }
        else if(a==3)
        {
            dq.Pop_Front();
            cout<<"Value removed successfully from front\nPress
Enter";
            getchar();getchar();
            clr
            menu();
        }
        else if(a==4)
        {
            dq.Pop_Back();
            cout<<"Value removed successfully from back\nPress
Enter";
            getchar();getchar();
            clr
            menu();
        }
        else if(a==5)
        {
            dq.Display();
            getchar();getchar();
            clr
            menu();
        }
        else
        {
            break;
        }
    }
}

```

#Deque Back operation:

**** Deque Menu ****

1. Enqueue (Front)
2. Enqueue (Back)
3. Dequeue (Front)
4. Dequeue (Back)
5. Display
6. Exit

Enter your option: 4

Value removed successfully from back
Press Enter

#Display operation:

**** Deque Menu ****

1. Enqueue (Front)
2. Enqueue (Back)
3. Dequeue (Front)
4. Dequeue (Back)
5. Display
6. Exit

Enter your option: 5

Front Element: 26

Rear Element: 55

Topic 4 [STL: list class]

Problem statement: Write a program to create and manipulate linked list using **list** class and its following methods to

- i) insert 8 integers using **push_back()** method
- ii) insert two elements using **push_front()** method
- iii) display all the elements of the list in forward direction with user-defined Display() method using **begin()** and **end()** methods and iterator
- iv) display all the elements of the list in reverse direction with a user-defined DisplayRev() method using **rbegin()** and **rend()** method and iterator
- v) display front element using **front()** method
- vi) display back element using **back()** method
- vii) delete front element using **pop_back()** method
- viii) delete front element using **pop_front()** method
- ix) search an element **x** using **find()** method
- x) insert a new element **x** before an existing element **y** using **insert()** method
- xi) insert a new element **x** after an existing element **y** using **insert()** method
- xii) count a particular element **x**
- xiii) count a elements with condition using predicate function
- xiv) delete a particular element **x** with **erase()** method
- xv) delete first 4 elements with **erase()** method
- xvi) delete a particular element **x** with **remove()** method
- xvii) delete a elements with condition using **remove_if()** method using predicate function
- xviii) assign elements from another list using **assign()** method
- xix) assign elements from an array using **assign()** method
- xx) sort the list using **sort()** method
- xxi) Delete consecutive similar elements using **unique()** method

Code	Output
<pre>#include<bits/stdc++.h> using namespace std; #define ll long long int #define str string #define ev(x) x%2==0 #define od(x) x%2==1 #define mod0(x,y) x%y==0 #define P_B push_back #define p_b pop_back #define P_F push_front #define p_f pop_front #define yes_or_no cout<<"YES ":cout<<"NO " #define newl cout<<endl int Even_count(list<int>&li) { int x=0; for(auto i:li) { if(ev(i)) {x++;} } return x; } bool is_odd(int x) { return od(x); } class List { private: list<int>ax; public: List() { } void Push_Back(int x) { ax.P_B(x); } void Push_Front(int x) { ax.P_F(x); } void Pop_Back()</pre>	

```

{ ax.p_b(); }
void Pop_Front()
{ ax.p_f(); }
void Display()
{
    if(ax.size()==0)
    { cout<<"NO ELEMENT\n";
      return; }
    auto i=ax.begin(); auto j=ax.end();
    while(i!=j)
    { cout<<*i<<" "; i++; }
    newl; }
void DisplayRev()
{
    if(ax.size()==0)
    { cout<<"NO ELEMENT\n";
      return; }
    auto i=ax.rbegin();j=ax.rend();
    while(i!=j)
    { cout<<*i<<" "; i++; }
    newl; }
int Front_show()
{
    if (ax.size()==0)
    { cout<<"NO ELEMENT";newl;
      return 0; }
    return ax.front();
}
void Back_show()
{
    if (ax.size()==0)
    { cout<<"NO ELEMENT";newl;
      return ; }
    cout<<"Rear Element: "<<ax.back();newl;
}
int Find(int x)
{ int y=(ax.begin(),ax.end(),x);
  return y; }
void Insert(int i,int x)
{ auto it=ax.begin();advance(it,i);
  ax.insert(it,x);Display(); }
void Count(int x)
{
    if(Find(x)==0)
    { cout<<x<<"is not found\n";return; }
    cout<<count(ax.begin(),ax.end(),x);newl;
}
void Even()
{ cout<<Even_count(ax);newl; }
void Delete(int i)
{ auto z=ax.begin();advance(z,i);
  ax.erase(z);Display(); }
void Sort()
{ ax.sort(); }
void Unique()
{ auto it=unique(ax.begin(),ax.end());
  ax.erase(it,ax.end());
  Display(); }
void Remove(int x)
{ auto b=remove(ax.begin(),ax.end(),x);
  ax.erase(b,ax.end());
  Display(); }
void Remove_If()
{ auto a=remove_if(ax.begin(),ax.end(),is_odd);
  ax.erase(a,ax.end());
  Display(); }
void Assign(list<int>B)
{ ax.assign(B.begin(),B.end());
  Display(); }

```

<pre> void Array_assign(int A[],int n) { ax.assign(A,A+n); } }; int main() { int A[]={10,64,25,36,78,64,69,70,64,67,67,62,62}; int n=13; List li; for(int i=1;i<=8;i++) { li.Push_Back(A[i-1]); } li.Display(); // i) li.Push_Front(A[8]);li.Push_Front(A[9]); li.Display(); // ii) li.Display(); // iii) li.DisplayRev(); // iv) cout<<"Front Element: "; cout<<li.Front_show();newl; // v) li.Back_show(); // vi) li.Pop_Front();li.Display(); // vii) li.Pop_Back();li.Display(); // viii) li.Find(64)?yes_or_no;li.Find(50)?yes_or_no; newl; // ix) li.Insert(2,75); // x) li.Insert(4,110); // xi) li.Count(64); // xii) li.Even(); // xiii) li.Delete(36); // xiv) for(int i=1;i<=4;i++) { li.Delete(li.Front_show()); } // xv) li.Remove(78); // xvi) li.Remove_If(); // xvii) list<int>B={15,25,51,45,60,80,64}; li.Assign(B); // xviii) li.Array_assign(A,n); // xix) li.Sort();li.Display(); // xx) li.Unique(); // xxi) } </pre>	<pre> i) 10 64 25 36 78 64 69 70 ii) 67 64 10 64 25 36 78 64 69 70 iii) 67 64 10 64 25 36 78 64 69 70 iv) 70 69 64 78 36 25 64 10 64 67 v) Front Element: 67 vi) Rear Element: 70 vii) 64 10 64 25 36 78 64 69 70 viii) 64 10 64 25 36 78 64 69 ix) YES NO x) 64 10 75 64 25 36 78 64 69 xi) 64 10 75 64 110 25 36 78 64 69 xii) 3 xiii) 7 xiv) 64 10 75 64 110 25 78 64 69 xv) 10 75 64 110 25 78 64 69 75 64 110 25 78 64 69 64 110 25 78 64 69 110 25 78 64 69 xvi) 110 25 64 69 xvii) 110 64 xviii) 15 25 51 45 60 80 64 xix) 10 64 25 36 78 64 69 70 64 67 67 62 62 xx) 10 25 36 62 62 64 64 64 67 67 69 70 78 xxi) 10 25 36 62 64 67 69 70 78 </pre>
---	--